

Trabalho 1 - Listas e análise de complexidade

Técnicas de Programação Avançada

INSTITUIÇÃO	Instituto Federal do Espírito Santo - Campus Serra
CURSO	Sistemas de Informação
PROFESSOR	Victorio Albani de Carvalho
INTEGRANTES	Bernardo Simão Rosa, Levi Monteiro e Matheus Abreu
REPOSITÓRIO	github.com/bs20242/tpa-trabalho1-etapa1

1. Desenvolvimento do trabalho

1.1 Participação dos integrantes

Integrante	Atividades descritas pelo grupo
Bernardo Simão Rosa	Lista genérica, nós, interface e testes da biblioteca. Relatório.
Matheus Abreu	Classe Contato, menu e regras do cadastro. Relatório.
Levi Monteiro	Gerador de contatos, testes de desempenho, análise de complexidade e gráficos. Relatório.

1.2 Como o programa funciona

A biblioteca usa nós ligados entre si. Cada nó guarda um valor do tipo T e a referência para o próximo nó. O construtor da lista recebe um comparador e um booleano que escolhe o modo ordenado ou não ordenado.

O programa pergunta qual modo usar e cria duas listas: uma compara nomes e outra compara telefones. As variáveis são do tipo `IColecao<Contato>`. As duas listas guardam referências para os mesmos objetos Contato, sem criar uma cópia do contato para cada lista.

O menu tem as sete opções pedidas: carregar arquivo, adicionar, buscar por nome, buscar por telefone, remover por telefone, alterar e sair. A carga usa `entrada.txt` como caminho padrão. Cada linha deve ter o formato `nome;telefone`, em UTF-8. Linhas inválidas, campos vazios e telefones já cadastrados não são inseridos.

`CadastroContatos` reúne as regras usadas pelo menu e pela carga do benchmark. Mantém um `HashSet` apenas com os telefones para evitar repetição sem fazer uma busca linear a cada cadastro. Essa regra fica fora da biblioteca. Nomes repetidos são permitidos, e a busca por nome devolve o primeiro contato encontrado.

Para remover uma pessoa sem apagar um homônimo, a lista por telefone usa `remove`, e a lista por nome usa `removeReferencia`. Esse método auxiliar procura a mesma instância com `==`. Ele não depende do nome nem de regras sobre contatos. A interface `IColecao` permanece igual à fornecida na disciplina. Os métodos auxiliares `removeReferencia` e `obterUltimo` ficam em `ListaEncadeada`; `CadastroContatos` acessa esses métodos nas listas que ele próprio criou.

Na alteração, o programa localiza um contato pelo nome, confere se o novo telefone está livre, remove o objeto antigo das duas listas e adiciona o novo objeto nas duas. Isso mantém a ordem correta. Ao sair, `quantidadeNos` informa o total atual.

O tempo da carga inclui leitura, validação e inserção nas duas listas. Nas buscas, mede-se a chamada de pesquisar. Na remoção, mede-se apenas remover na lista por telefone; localizar o contato e atualizar a lista por nome ficam fora desse intervalo. O menu deixa claro qual tempo está mostrando. A opção de alteração não mede tempo, pois isso não foi pedido.

`Contato.toString` devolve nome e telefone separados por hífen. `ListaEncadeada.toString` coloca os valores entre colchetes e os separa por vírgulas. A biblioteca não imprime mensagens para o usuário.

1.3 Utilização de Ferramentas de Inteligência Artificial

O grupo usou ferramentas de Inteligência Artificial apenas como apoio em dúvidas técnicas e para ajudar em algumas tarefas:

- **Geração dos dados de teste:** consulta sobre como gerar muitos telefones sem repetição em Java, usando HashSet e uma seed fixa.
- **Gráficos:** ajuda com a sintaxe do matplotlib, em Python, para formatar as curvas e os eixos.
- **Conferência das contas:** os integrantes fizeram a contagem das operações de cada linha com base no Modelo Simplificado de Custos. A IA ajudou a conferir as simplificações das somas.
- **Formatação do texto:** ajuda para organizar as tabelas em Markdown.

Autoria e compreensão: os integrantes tomaram as decisões sobre a biblioteca, escreveram todas as classes, executaram os testes de desempenho em laboratório e fizeram a análise teórica. O grupo compreendeu e conferiu os resultados, e todos estão preparados para explicar o trabalho na entrevista individual.

1.4 Organização e execução

O código fica em `src/colecao`, `src/dominio`, `src/app` e `src/testes`. A pasta `dados` guarda os scripts, os tempos e os dados do ambiente. A pasta `relatorio` guarda os gráficos e o PDF. `README.md` traz os comandos completos para compilar, testar e repetir as medições.

É necessário JDK 11 ou mais recente. Os testes da lista precisam de `-ea`; se essa opção faltar, o programa de testes avisa e termina com erro. Os testes adicionais verificam nomes repetidos, telefones duplicados, carga repetida e manutenção das duas listas.

2. Análise de complexidade

2.1 Como fazemos a contagem

n é a quantidade de nós antes de uma operação. N é a quantidade de contatos do arquivo. Para comparar os algoritmos, consideramos que ler ou alterar uma referência, testar uma condição simples e criar um nó têm custo constante.

Cada c com um número representa o custo constante da linha indicada, e A reúne os custos que não dependem de n . Não atribuímos uma quantidade exata de instruções de máquina a uma linha Java. As tabelas contam quantas vezes cada parte é executada e mostram como o custo cresce.

Nas tabelas de complexidade, consideramos o comparador com custo constante. Isso vale para analisar o número de nós visitados e para entradas com tamanho de chave limitado, como as usadas neste teste. Em geral, comparar strings pode custar $O(L)$, sendo L o comprimento comparado. Sem essa hipótese, os percursos com comparações podem custar $O(nL)$.

$O(1)$ significa custo limitado por uma constante. $O(n)$ indica um limite linear; $O(n^2)$, um limite quadrático. Quando o algoritmo necessariamente faz trabalho proporcional a n , também podemos escrever $\Theta(n)$.

Os códigos abaixo foram extraídos de `ListaEncadeada.java`, sem linhas vazias e com numeração própria para facilitar a leitura. Chaves e assinatura do método não acrescentam trabalho ao percurso.

2.2 quantidadeNos

```
1: public int quantidadeNos() {
2:     int total = 0;
3:     No<T> atual = primeiro;
4:     while (atual != null) {
5:         total++;
6:         atual = atual.getProximo();
7:     }
8:     return total;
9: }
```

Linhas	O que acontece	Quantidade de execuções
2 e 3	Iniciam o contador e a referência para o primeiro nó.	Uma vez cada.
4	Testa se ainda existe um nó.	$n + 1$ vezes.
5	Aumenta o contador.	n vezes.
6	Passa para o próximo nó.	n vezes.
8	Devolve o total.	Uma vez.

$$T(n) = c_2 + c_3 + (n + 1)c_4 + n(c_5 + c_6) + c_8 = A + Bn.$$

A lista inteira é percorrida em qualquer caso. Por isso, o melhor e o pior caso são $\Theta(n)$, tanto na lista ordenada quanto na não ordenada. Uma lista vazia executa apenas as partes constantes.

2.3 adicionar

```
1: public boolean adicionar(T novoValor) {
2:     if (novoValor == null) {
3:         return false;
4:     }
5:     No<T> novoNo = new No<>(novoValor);
6:     if (!ordenada) {
7:         novoNo.setProximo(primeiro);
8:         primeiro = novoNo;
9:         return true;
10:    }
11:    if (primeiro == null || comparador.compare(novoValor, primeiro.getValor()) <= 0) {
12:        novoNo.setProximo(primeiro);
13:        primeiro = novoNo;
14:        return true;
15:    }
16:    No<T> atual = primeiro;
17:    while (atual.getProximo() != null && comparador.compare(novoValor, atual.getProximo().getValor()) >
18:        0) {
19:        atual = atual.getProximo();
20:    }
21:    novoNo.setProximo(atual.getProximo());
22:    atual.setProximo(novoNo);
23:    return true;
24: }
```

Lista não ordenada: para um valor não nulo, a inserção é sempre no início.

Linhas	O que acontece	Quantidade de execuções
2	Verifica se o valor é nulo.	Uma vez.
3	Recusa valor nulo.	Zero para uma inserção válida.
5	Cria o nó.	Uma vez.
6	Escolhe o modo não ordenado.	Uma vez.
7 e 8	Liga o novo nó ao primeiro e atualiza o início.	Uma vez cada.
9	Devolve true.	Uma vez.
11 a 22	Parte exclusiva da inserção ordenada.	Zero neste modo.

$T(n) = c_2 + c_5 + c_6 + c_7 + c_8 + c_9$. O custo não depende de n : melhor e pior caso $O(1)$.

Lista ordenada: para n maior ou igual a 1, o pior caso ocorre ao inserir um valor maior que todos os existentes.

Linhas	O que acontece no pior caso	Quantidade de execuções
2, 5 e 6	Validam o valor, criam o nó e verificam o modo.	Uma vez cada.
3 e 7 a 9	Retornos que não são usados neste caso.	Zero.
11	Verifica se a inserção seria no início.	Uma vez.
12 a 14	Inserção no início, não usada neste caso.	Zero.
16	Começa o percurso no primeiro nó.	Uma vez.
17, primeira condição	Testa se existe próximo nó.	n vezes.

Linhas	O que acontece no pior caso	Quantidade de execuções
17, comparação	Compara com o próximo valor; não executa quando o próximo é nulo.	n - 1 vezes.
18	Avança um nó.	n - 1 vezes.
20, 21 e 22	Ligam o novo nó ao fim e retornam true.	Uma vez cada.

$$T(n) = A + n \cdot c_{17}a + (n - 1)(c_{17}b + c_{18}) = A' + Bn.$$

O pior caso é $O(n)$. O melhor caso é $O(1)$, quando a lista está vazia ou o valor entra antes do primeiro nó. Essa é a principal diferença entre os modos para uma inserção isolada.

2.4 pesquisar

```

1: public T pesquisar(T valor) {
2:     if (valor == null || primeiro == null) {
3:         return null;
4:     }
5:     No<T> atual = primeiro;
6:     if (ordenada) {
7:         while (atual != null) {
8:             int comp = comparador.compare(atual.getValor(), valor);
9:             if (comp == 0) {
10:                 return atual.getValor();
11:             }
12:             if (comp > 0) {
13:                 return null;
14:             }
15:             atual = atual.getProximo();
16:         }
17:     } else {
18:         while (atual != null) {
19:             if (comparador.compare(atual.getValor(), valor) == 0) {
20:                 return atual.getValor();
21:             }
22:             atual = atual.getProximo();
23:         }
24:     }
25:     return null;
26: }

```

Para contar o pior caso sem misturar saídas diferentes, usamos uma chave ausente e, no modo ordenado, maior que todas as chaves da lista. Consideramos n maior ou igual a 1.

Linhas	O que acontece	Quantidade de execuções
2, 5 e 6	Verificam a entrada, iniciam a referência e escolhem o modo.	Uma vez cada.
3	Retorno para entrada nula ou lista vazia.	Zero neste caso.
7	Condição do laço ordenado.	n + 1 no modo ordenado.
8, 9 e 12	Comparam e verificam igualdade ou possibilidade de parar.	n no modo ordenado.
10 e 13	Retornos antecipados.	Zero neste caso.
15	Avança a referência.	n no modo ordenado.
18	Condição do laço não ordenado.	n + 1 no modo não ordenado.
19 e 22	Comparam e avançam a referência.	n no modo não ordenado.

Linhas	O que acontece	Quantidade de execuções
20	Retorno ao encontrar a chave.	Zero neste caso.
25	Informa que não encontrou.	Uma vez.

Ordenada: $T(n) = A + (n + 1)c_7 + n(c_8 + c_9 + c_{12} + c_{15})$.

Não ordenada: $T(n) = A + (n + 1)c_{18} + n(c_{19} + c_{22})$.

Os dois custos são da forma $A' + Bn$, então o pior caso é $O(n)$ nos dois modos. Se a chave só aparece no último nó, o custo também é linear, mas o retorno acontece dentro do laço: não há a última condição falsa. Se existem chaves iguais antes da cauda, a busca termina na primeira delas.

O melhor caso é $O(1)$, quando o primeiro nó corresponde à chave. A ordenação permite parar antes ao encontrar um valor maior que a chave procurada; isso ajuda em algumas buscas sem resultado, mas não muda o pior caso.

2.5 remover

```

1: public boolean remover(T valor) {
2:     if (valor == null || primeiro == null) {
3:         return false;
4:     }
5:     if (comparador.compare(primeiro.getValor(), valor) == 0) {
6:         primeiro = primeiro.getProximo();
7:         return true;
8:     }
9:     if (ordenada && comparador.compare(primeiro.getValor(), valor) > 0) {
10:        return false;
11:    }
12:    No<T> anterior = primeiro;
13:    No<T> atual = primeiro.getProximo();
14:    while (atual != null) {
15:        int comp = comparador.compare(atual.getValor(), valor);
16:        if (comp == 0) {
17:            anterior.setProximo(atual.getProximo());
18:            return true;
19:        }
20:        if (ordenada && comp > 0) {
21:            return false;
22:        }
23:        anterior = atual;
24:        atual = atual.getProximo();
25:    }
26:    return false;
27: }
```

Usamos novamente uma chave ausente, maior que todas as existentes no modo ordenado, e n maior ou igual a 1. O primeiro nó é testado separadamente; o laço percorre os outros $n - 1$ nós.

Linhas	O que acontece	Quantidade de execuções
2	Verifica entrada e lista vazia.	Uma vez.
3	Retorna para entrada nula ou lista vazia.	Zero neste caso.
5	Compara com o primeiro nó.	Uma vez.
6 e 7	Removem o primeiro nó.	Zero neste caso.
9	Verifica parada antecipada; só compara valores se ordenada for true.	Uma vez.

Linhas	O que acontece	Quantidade de execuções
10	Retorno antecipado.	Zero neste caso.
12 e 13	Iniciam anterior e atual.	Uma vez cada.
14	Testa se atual existe.	n vezes.
15, 16 e 20	Comparam e verificam remoção ou parada antecipada.	n - 1 vezes cada.
17, 18 e 21	Religamento ou retornos antecipados.	Zero neste caso.
23 e 24	Atualizam anterior e atual.	n - 1 vezes cada.
26	Informa que não removeu.	Uma vez.

$$T(n) = A + n c_{14} + (n - 1)(c_{15} + c_{16} + c_{20} + c_{23} + c_{24}) = A' + Bn.$$

O custo de c_9 e c_{20} varia entre os modos por causa do &&, mas continua constante. O pior caso é $O(n)$ nos dois modos. Remover uma chave que só aparece na cauda também é linear: o algoritmo percorre até encontrá-la, religa os nós e retorna. Nesse caso, não chega à última condição falsa nem ao retorno da linha 26.

O melhor caso é $O(1)$, ao remover o primeiro nó. A ordenação pode encerrar algumas tentativas sem resultado antes do fim, sem alterar o pior caso.

2.6 Resumo e custo do cadastro completo

Método	Não ordenada: melhor	Não ordenada: pior	Ordenada: melhor	Ordenada: pior
adicionar	$O(1)$	$O(1)$	$O(1)$	$O(n)$
pesquisar	$O(1)$	$O(n)$	$O(1)$	$O(n)$
remover	$O(1)$	$O(n)$	$O(1)$	$O(n)$
quantidadeNos	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

O método auxiliar `removerReferencia` também percorre os nós: custa $O(1)$ se o objeto está no início e $O(n)$ no pior caso. `obterUltimo` custa $\Theta(n)$ para uma lista não vazia. Esses métodos não criam cópias dos contatos.

Inserir na lista não é o mesmo que cadastrar um contato. `CadastroContatos.adicionar` usa um `HashSet` para verificar o telefone em tempo esperado $O(1)$ e depois insere o objeto nas duas listas. A regra de telefone único continua fora da biblioteca.

Na carga de N telefones únicos, a validação com `HashSet` tem custo esperado de $\Theta(N)$ no total. Assim, no modo não ordenado, a carga completa tem custo esperado de $\Theta(N)$: a validação (esperada $O(1)$) e as duas inserções têm custo constante por contato.

Na carga ordenada, a validação continua com custo esperado $\Theta(N)$, mas as inserções ordenadas podem percorrer uma lista que cresce até N . Por isso, o custo total é $O(N^2)$ no pior caso. Para chaves aleatórias, uma inserção costuma percorrer uma parte da lista, mas isso não é uma contagem do pior caso.

3. Testes de desempenho

3.1 Como os testes foram feitos

Foram usados quatro tamanhos: 10.000, 25.000, 50.000 e 100.000 contatos. Cada tamanho foi executado nos dois modos, com três repetições por modo, totalizando 24 processos Java. Cada processo começa com listas vazias e lê o arquivo completo. O CSV resumido mostra a mediana das três medições de cada operação, isto é, o valor do meio depois de ordenar os três tempos.

O gerador usa seed 42 e garante telefones únicos. O primeiro contato é ZZZ Contato Final, com telefone artificial 9999999999. Os outros nomes são sorteados e podem se repetir; nenhum deles é igual ao nome reservado. Seus telefones começam com DDD 27, 28 ou 29.

Esse contato reservado fica no fim das duas listas nos dois modos: na não ordenada, por ser o primeiro inserido; na ordenada, por ter nome e telefone maiores que os demais. Assim, as duas buscas e a remoção por telefone usam o mesmo contato e precisam alcançar a cauda. Esse dado foi escolhido para testar o pior caso, não para representar uma pessoa real.

Antes de cronometrar as buscas, o benchmark verifica se o alvo é a cauda das duas listas e se a pesquisa por nome devolve essa mesma instância. A seleção e essas verificações não entram no tempo. Elas fazem percursos prévios; por isso, não tratamos os resultados como testes de memória ou JVM completamente frias.

As quatro medições são:

- Leitura e montagem: chamada de `CadastroContatos.carregar`, igual à usada pelo menu. Inclui validar telefones e inserir nas duas listas.
- Busca por telefone: apenas a chamada de pesquisar na lista por telefone.
- Busca por nome: apenas a chamada de pesquisar na lista por nome.
- Remoção por telefone: apenas a chamada de remover na lista por telefone. A retirada do mesmo objeto da lista por nome ocorre depois, fora do tempo, como no menu.

As chaves de busca são criadas antes de iniciar o cronômetro. Depois das medições, o benchmark confere os resultados e a quantidade de contatos nas duas listas. Um resultado incorreto interrompe a execução.

Os tempos vêm de `System.nanoTime` e são convertidos para milissegundos. Não houve aquecimento controlado nem isolamento do sistema operacional. São medições simples para observar o crescimento, e não uma prova experimental da complexidade.

Ambiente registrado nesta execução: Windows-11-10.0.26200-SP0. Intel64 Family 6 Model 165 Stepping 5, GenuineIntel. Java: openjdk version "21.0.6" 2025-01-21 LTS; OpenJDK Runtime Environment Corretto-21.0.6.7.1 (build 21.0.6+7-LTS); OpenJDK 64-Bit Server VM Corretto-21.0.6.7.1 (build 21.0.6+7-LTS, mixed mode, sharing).

Início da coleta (UTC): 2026-09-14T23:51:39.078657+00:00. Os detalhes estão em `dados/ambiente.json`, junto com a seed e os hashes SHA-256 dos arquivos. `dados/resultados_brutos.csv` guarda as 24 linhas originais; `dados/resultados.csv` guarda as oito linhas com medianas. `dados/executar_benchmarks.py` permite repetir a coleta.

3.2 Tempos medidos

Tabela 1 - Leitura e montagem completa das duas listas (ms; mediana de três execuções)

N	Não ordenada	Ordenada
10.000	34,4864	961,5603
25.000	59,1761	8590,5368
50.000	89,2657	65166,8602
100.000	145,9843	252445,4744

Tabela 2 - Busca do contato da cauda (ms; mediana de três execuções)

N	Não ord.: telefone	Não ord.: nome	Ord.: telefone	Ord.: nome
10.000	1,9462	1,3518	0,5913	0,5995
25.000	4,1928	3,8690	2,8232	3,3659
50.000	6,9765	2,1509	5,6915	5,1778
100.000	6,0489	2,0732	10,6781	9,3485

Tabela 3 - Remoção na lista por telefone (ms; mediana de três execuções)

N	Não ordenada	Ordenada
10.000	1,0386	0,7271
25.000	3,9615	3,9314
50.000	7,6943	9,1309
100.000	8,4620	16,6710

3.3 Gráficos

Gráfico 1 - Leitura e montagem completa

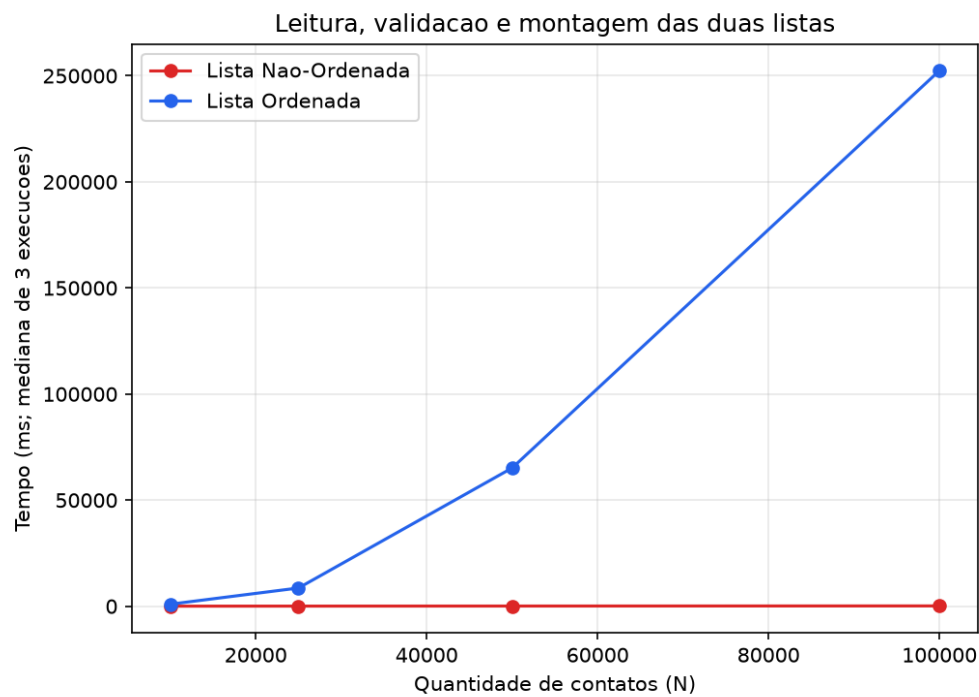


Gráfico 2 - Busca do contato da cauda

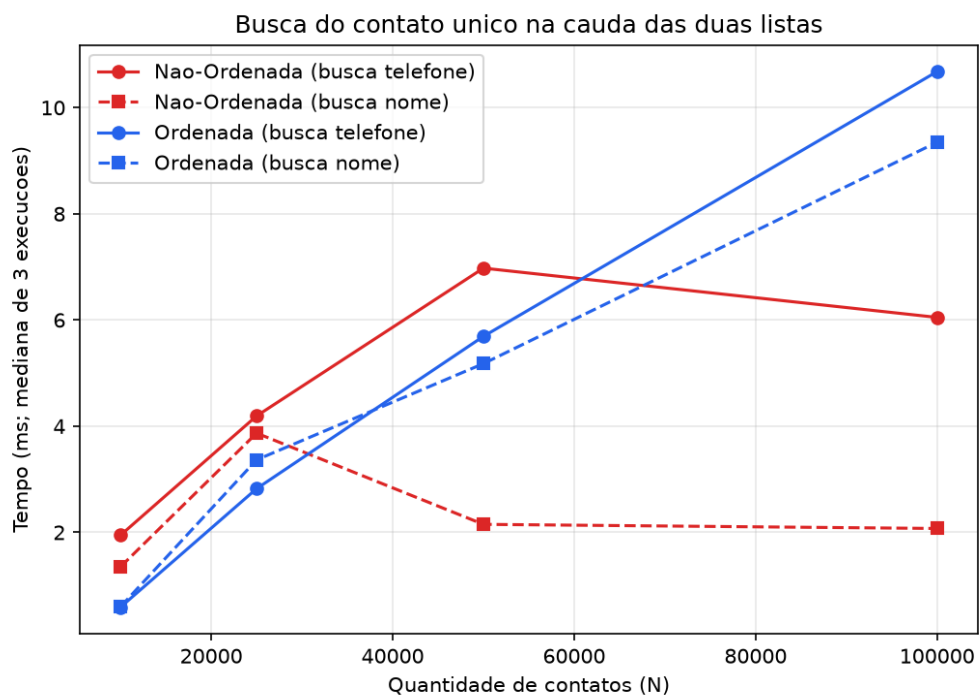


Gráfico 3 - Remoção na lista por telefone



3.4 O que os resultados mostram

- Montagem não ordenada: ao passar de 50.000 para 100.000 contatos, o tempo mediano aumentou 1.64 vezes.
- Montagem ordenada: ao passar de 50.000 para 100.000 contatos, o tempo mediano aumentou 3.87 vezes.

Em um modelo quadrático simples, dobrar N multiplica a parte dominante do custo por cerca de quatro. A comparação com essa referência ajuda a interpretar os tempos, mas não exige uma razão exata: alocação, acesso à memória e execução da JVM também influenciam o tempo observado.

A carga não ordenada tem crescimento aproximadamente linear, porque a validação de telefone usa um HashSet em tempo esperado $O(1)$ e cada inserção no início também custa $O(1)$. Os tempos da Tabela 1 sobem de 34,49 ms para 145,98 ms entre 10.000 e 100.000 contatos, um aumento de 4,23 vezes quando N aumenta 10 vezes. Na carga ordenada, além da validação, é necessário procurar a posição de inserção; por isso, o custo dominante é quadrático.

Nas buscas, o alvo é único e está na cauda: são comparados N valores. Na remoção por telefone, é preciso chegar ao último nó antes de retirá-lo. Esse percurso explica o custo linear dessas operações. O menor tempo de uma execução não significa que ela tenha visitado menos nós.

As medições de busca e remoção são curtas e variam entre as repetições. Três repetições e a mediana reduzem o peso de um resultado isolado, mas ainda são uma amostra pequena. Não é possível atribuir uma diferença específica ao JIT, ao coletor de lixo ou a outro processo sem medir esses fatores. Os dados brutos permitem conferir essa variação.

Conclusão: a biblioteca tem inserção constante no modo não ordenado e linear no pior caso ordenado. Busca e remoção têm pior caso linear nos dois modos; quantidadeNos sempre percorre a lista. No cadastro completo, a validação dos telefones usa HashSet: o modo não ordenado tem custo esperado $\Theta(N)$, enquanto o modo ordenado tem custo $O(N^2)$ por causa da busca da posição de inserção. As medições confirmam esse comportamento geral, considerando a variação normal da JVM.